

Technical manual

Architecture, data model, API, security and deployment.

Police Bagage is a boarding and baggage fraud prevention system, deployed at Kinshasa International Airport (FIH) on Air Congo flights.

The problem it solves: at the check-in desk, a baggage tag can be issued for a passenger who declared no baggage. The parcel then travels in the hold without any control. The system ties every bag to a genuinely registered passenger and intercepts undeclared parcels before loading, alerting the supervisor immediately.

This document is aimed at developers and operators. For day to day use of the portals, see the user manuals.

1. Architecture

The project is a Turborepo monorepo holding five applications and three shared packages.

apps/		
mobile/	React Native + Expo	field agents on Zebra handhelds
web/	Next.js	supervisor and admin dashboard
vols/	Next.js	public flight board
litige/	Next.js	baggage dispute handling
tracking/	Next.js	public passenger baggage tracking
packages/		
api/	Fastify	anti-fraud logic, scan routes
bcbp-parser/	TypeScript	boarding pass and tag parser
shared/	TypeScript	shared types and constants
supabase/		
migrations/	SQL	schema, RLS, realtime, functions

Technical layers

Layer	Technology	Note
Mobile	React Native 0.76, Expo SDK 52	Android only, Zebra handhelds
Web	Next.js 15 App Router	Four separate portals
API	Node 20+, Fastify 5	Single process, service_role key
Database, Auth, Realtime	Supabase (PostgreSQL 17)	RLS enabled on every table
BCBP parser	npm package bcbp v6	IATA standard
Tag parser	in-house function	10 or 13 digit format

No language model dependency. BCBP is a documented standard, and the baggage tag is parsed by an in-house function.

a

2. Data model

Eight tables in the public schema. The columns that drive business logic are listed below.

Table	Purpose	Key columns
profiles	Accounts and roles	role (admin, supervisor, agent), gate, airport_code
flights	Flights of the day	flight_number, origin, destination, stops, status, date
passengers	Registered passengers	pnr, seat, declared_baggage_count, boarded
passenger_legs	Stopovers per passenger	origin, destination, leg_order
baggage	Bags	tag_number, serial_number, is_confirmed, on_dolly, in_hold, rush, soute
fraud_alerts	Rejection alerts	tag_number, reason, gate, resolved
baggage_disputes	Disputes and claims	status, from_passenger, notes
airline_codes	Airline codes	numeric_code, iata_code

Passenger to baggage link

The link is never made on the airline code. Numeric code 071 maps to both Ethiopian Airlines and Air Congo, which is expected and correct. The key is always the 6 digit serial number, combined with the flight and the date.

```

4 071 303791 002
| |      |    +-- 3 digits : number of consecutive bags (boarding pass tag)
| |      +----- 6 digits : serial number, the linking key
| +----- 3 digits : airline numeric code
+----- 1 digit  : Baggage Tag Issuer Code

```

The physical tag scanned at the belt is 10 digits. The tag carried by the boarding pass is 13: the last three say how many consecutive bags are covered. At check-in the API expands that range into the matching physical tags.

a

3. Scan API

Seven routes exposed by the Fastify service. Every scan route requires a valid authentication token.

Route	Use	Effect
POST /scan/boarding	Passenger check-in	Creates the passenger, legs, and pre-registers their bags
POST /scan/baggage	Tag at the belt	Applies the 5 anti-fraud rules, confirms or rejects
POST /scan/dolly	X-ray checkpoint	Accepts confirmed bags only, returns progress
POST /scan/soute	Aircraft hold	Marks the bag in the forward or aft hold
POST /scan/rush	Left behind bag	Marks it for rerouting on the next flight
POST /scan/load-all	Bulk loading	Pushes every confirmed non rush bag into the hold
POST /scan/embarquement	Boarding gate	Marks the passenger boarded, returns remaining count

GET /health

Monitoring

Availability probe, no authentication

Authentication

The API runs with the `service_role` key, which bypasses RLS. It therefore authenticates every call itself. A Fastify hook validates the agent Supabase token, checks their role, then decorates the request with the verified identity.

The scanner identity is derived from the token, never from the request body. A client cannot impersonate another agent in the audit trail.

```
Authorization: Bearer <Supabase access_token>
```

```
401 token missing, invalid or expired
```

```
403 profile not found, or role not allowed
```

a

4. Anti-fraud rules

Five rejection rules, applied when a tag is scanned at the belt. They live in a pure function with no database access, which makes them testable on their own. The route gathers the context from the database, then delegates the decision.

Rule	Condition	Outcome
1	No pre-registered bag for this serial number on this flight	Reject and raise a fraud alert
2	The passenger declared 0 bags	Reject and raise a fraud alert
3	Quota reached: confirmed count at or above declared count	Reject and raise a fraud alert
4	Tag already confirmed on this flight	Plain reject, no alert
5	The bag belongs to another flight	Plain reject, no alert

Rules 4 and 5 raise no alert: a double scan or a wrong gate is not fraud. Only rules 1 to 3 write to `fraud_alerts`, with de-duplication per tag and per flight.

Responsibility

The baggage agent is never at fault: they scan what reaches the belt. Fraud originates at the check-in desk, where a tag was printed against a booking with no declared bag. The system intercepts the parcel before the hold and alerts the supervisor so they can act physically.

a

5. Baggage life cycle

Each step maps to its own scanning station and to a database column.

1. Check-in	boarding pass scanned	-> bags pre-registered (<code>is_confirmed = false</code>)
2. Baggage belt	tag scanned	-> <code>is_confirmed = true</code> , or anti-fraud reject
3. Dolly (X-ray)	tag scanned	-> <code>on_dolly = true</code> , only if already confirmed
4. Hold position	tag scanned	-> <code>soute = forward aft</code>

5. Loading	bulk action	-> in_hold = true for non rush bags
Rush	tag scanned	-> rush = true, rerouted to the next flight

The Dolly station

The dolly is the cart that carries screened bags to the aircraft. As bags leave the X-ray, the agent scans each one. Only bags already confirmed at the belt are accepted: any unknown or unregistered bag is refused. The screen shows progress as a counter and reports the dolly complete once the number of loaded bags matches the number of registered bags for the flight.

a

6. Security

Roles

Role	Platform	Permissions
agent	Mobile only	Scan boarding passes and tags
supervisor	Web	Dashboard, baggage, reports, disputes
admin	Web	Everything, including creating and deleting accounts

Row Level Security

RLS is enabled on all eight tables. Policies rely on an `auth_role()` function declared SECURITY DEFINER, which reads the role without triggering recursion on the profiles policies. That function must remain executable by the authenticated role, otherwise every policy stops working.

Account creation

Public sign-up is disabled. Accounts are created only by an administrator, through a server route that checks their role in the database before using the admin key. A trigger populates profiles from the metadata, restricting the role to the three valid values.

Things to watch

- The `service_role` key must never reach the client, only the API and server routes.
- Public tracking and claim endpoints are rate limited per IP address.
- Public portals return no passenger data without a valid PNR or tag number.

a

7. Realtime and load

The passengers, baggage, fraud_alerts and flights tables are published to realtime, with `REPLICA IDENTITY FULL` so that update events also carry the previous column values.

Incremental counting on mobile

The mobile app does not refetch counters on every event. It applies a local delta from the old and new values it receives. A scan performed by one agent therefore triggers no query at all on the other connected devices.

At startup, statistics for every flight of the day come from a single aggregated SQL function, instead of a series of per flight count queries.

```
select * from public.flight_stats_for_date(current_date);

flight_id | pax | bag_total | bag_ok | boarded
```

Flight board cache

The public flight portal serves the same list to every visitor. A 20 second in memory cache, with de-duplication of concurrent requests, brings the load down to one query per date and per interval whatever the number of visitors. Airport filtering happens on the client, which preserves that shared cache.

a

8. Deployment

The monorepo is the source of truth. Deployment targets are separate standalone repositories, fed by copying.

Repository	Content	Deployment
Police	Full monorepo	Source of truth, deploys nothing
API-POLICE	Standalone API, server.js bundle	Hostinger, auto-deploy on push
police-web	Supervisor dashboard	Web hosting
police-vols	Flight board	Web hosting
police-litige	Dispute handling	Web hosting
police-tracking	Baggage tracking	Web hosting

Changing the monorepo propagates nothing on its own. Sources must be copied into the matching standalone repository, rebuilt if needed, then pushed.

Mobile application

The mobile app has no over the air update. Any change requires a full rebuild and a reinstall on every handheld.

```
cd apps/mobile
eas build -p android --profile production
```

Worth knowing: deploying an API that requires something new from the client breaks scanning on handhelds that are not updated yet. Update the mobile app first, the API second.

a

9. Useful commands

npm install	install monorepo dependencies
npm run dev -w apps/web	run a portal in development
npm run typecheck -w @police/api	type check a package
npm test -w @police/api	run the tests
npx tsc --noEmit	type check a standalone repository

<code>node docs/capture.mjs --login</code>	screenshots for the manuals
<code>python docs/manuels_contenu.py</code>	build the user manuals
<code>python docs/technique_en.py</code>	build this technical manual